

Day 2 — Linear Search and Array Searching Patterns

Challenge: 30 Days DSA — Zero to Intermediate **Language:** JavaScript **Topic:** Linear Search and Array Searching Patterns

Concepts Covered

1. Linear Search
 2. Searching an element in an array
 3. Finding the index of a target
 4. Dynamic target variable
 5. Using `-1` when an element is not found
 6. Why and when to use `break`
 7. Counting occurrences
 8. Finding the first occurrence
 9. Finding the last occurrence
 10. Finding the index of the maximum value
 11. Reusable functions for DSA problems
 12. Time and Space Complexity
-

What is Linear Search?

Linear Search is the simplest searching algorithm. It checks **every element** of an array, one by one, from the start to the end, until it finds the target value or reaches the end of the array.

Think of it like looking for your friend's name in a class attendance

register — you go line by line until you find it (or reach the end and realize the name isn't there).

Core Patterns Used Throughout Day 2

Pattern	Meaning
<code>index = -1</code>	Default value used when the element is not found
<code>index = i + break</code>	Stop at the first occurrence and exit the loop immediately
<code>index = i</code> without <code>break</code>	Keep looping so the last match overwrites earlier ones → last occurrence
<code>count++</code>	Increment a counter every time a match is found → counting occurrences
<code>maxIndex</code>	Store the index of the largest value seen so far while scanning
<code>target variable</code>	Instead of hardcoding the value to search for, store it in a variable so the function works for any input

Problem 1: Find the Index of a Target Using Linear Search

Problem Statement

Given an array of numbers and a target value, find the index of the target. If the target does not exist in the array, return `-1`.

Approach

Loop through the array from index 0 to the last index. Compare each element with the target. If a match is found, return that index immediately.

Simple Explanation

We check each box (element) one at a time and ask, "Is this the one I'm looking for?" As soon as we find it, we stop and report where it is. If we never find it, we say "not found" using `-1`.

Dry Run

Array: `[4, 2, 7, 1, 9]`, target = `7`

i	arr[i]	Match?
0	4	No
1	2	No
2	7	Yes → return 2

Output: `2`

JavaScript Solution

```
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
      return i; // found - return index
    }
  }
  return -1; // not found
}
```

```
console.log(linearSearch([4, 2, 7, 1, 9], 7)); // 2
console.log(linearSearch([4, 2, 7, 1, 9], 5)); // -1
```

Time Complexity

$O(n)$ — in the worst case, we check every element once.

Space Complexity

$O(1)$ — no extra space is used apart from a few variables.

Key Learning

The `-1` return value is a universal convention in searching problems to represent "not found." Always handle this case explicitly.

Problem 2: Search Using a Dynamic Target

Problem Statement

Improve the previous solution so the target value isn't hardcoded — the function should accept **any** target as an argument and work for any array.

Approach

Instead of writing a new function for every value you want to search, pass the target as a parameter. The loop logic stays the same, but `target` becomes a variable supplied by the caller.

Simple Explanation

Rather than building a separate search function for "find 5", "find 10", "find 99", we build **one** function that takes whatever number we want to search as input. This is the foundation of writing reusable code.

Dry Run

Array: [10, 20, 30, 40], target = 30

i	arr[i]	Match?
0	10	No
1	20	No
2	30	Yes → return 2

Output: 2

JavaScript Solution

```
function searchDynamicTarget(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      return i;  
    }  
  }  
  return -1;  
}
```

```
console.log(searchDynamicTarget([10, 20, 30, 40], 30)); //  
console.log(searchDynamicTarget([10, 20, 30, 40], 100));
```

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Key Learning

Making the `target` a parameter (instead of a hardcoded value) turns a one-off script into a reusable tool — a mindset shift that matters a lot in real DSA problem-solving.

Problem 3: Count Occurrences of a Target

Problem Statement

Given an array and a target value, count how many times the target appears in the array.

Approach

Loop through the entire array. Every time the current element equals the target, increase a counter. **Do not** `break`, because stopping early would miss later occurrences.

Simple Explanation

We're not looking for just one match — we want to know the total number of matches. So we must check every single element, no matter how many times we've already found the target.

Dry Run

Array: [2, 5, 2, 8, 2, 1], target = 2

i	arr[i]	Match?	count
0	2	Yes	1
1	5	No	1
2	2	Yes	2
3	8	No	2
4	2	Yes	3
5	1	No	3

Output: 3

JavaScript Solution

```
function countOccurrences(arr, target) {  
  let count = 0;  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      count++;  
    }  
  }  
  return count;  
}  
  
console.log(countOccurrences([2, 5, 2, 8, 2, 1], 2)); //
```

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Key Learning

`break` is intentionally **not used** here — counting requires visiting every element, not stopping at the first match.

Problem 4: Find the First Occurrence of a Target

Problem Statement

Given an array with possible duplicate values, find the index of the **first** time the target appears.

Approach

Loop through the array from the start. The moment a match is found, that index is guaranteed to be the first occurrence — so return/`break` immediately.

Simple Explanation

Since we're scanning left to right, the very first match we encounter is automatically the earliest one. There's no reason to keep looking, so we stop right away.

Dry Run

Array: `[3, 7, 7, 9, 7]`, target = 7

i	arr[i]	Match?
0	3	No
1	7	Yes → break, return 1

Output: 1

JavaScript Solution

```
function firstOccurrence(arr, target) {
  let index = -1;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) {
      index = i;
      break; // stop immediately — this is the first match
    }
  }
  return index;
}

console.log(firstOccurrence([3, 7, 7, 9, 7], 7)); // 1
```

Time Complexity

$O(n)$ worst case (target near the end or absent)

Space Complexity

$O(1)$

Key Learning

`break` is used here because the first match found while scanning left to right **is** the first occurrence — continuing the loop would waste time and could never produce a "better" answer.

Problem 5: Find the Last Occurrence of a Target

Problem Statement

Given an array with possible duplicate values, find the index of the **last** time the target appears.

Approach

Loop through the entire array without stopping. Every time a match is found, update `index` to the current position. Since we never break, the final update will belong to the last match.

Simple Explanation

We keep overwriting our "last known position" every time we see the target again. By the time the loop finishes, whatever value is stored is the last one we saw.

Dry Run

Array: [3, 7, 7, 9, 7], target = 7

i	arr[i]	Match?	index
0	3	No	-1
1	7	Yes	1
2	7	Yes	2
3	9	No	2

4	7	Yes	4
---	---	-----	---

Output: 4

JavaScript Solution

```
function lastOccurrence(arr, target) {  
  let index = -1;  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) {  
      index = i; // keep updating, no break  
    }  
  }  
  return index;  
}  
  
console.log(lastOccurrence([3, 7, 7, 9, 7], 7)); // 4
```

Time Complexity

$O(n)$ — the full array must always be scanned.

Space Complexity

$O(1)$

Key Learning

`break` is intentionally **avoided** here. We must visit every element to guarantee that the last update to `index` truly corresponds to the last occurrence.

Problem 6: Find the Index of the First Occurrence of the Maximum Value

Problem Statement

Given an array of numbers, find the index of the maximum value. If the maximum value appears multiple times, return the index of its **first** occurrence.

Approach

Assume the first element is the maximum and store its index in `maxIndex`. Loop through the rest of the array; whenever a strictly greater value is found, update `maxIndex`. Using strictly `>` (not `>=`) ensures later equal values don't overwrite the first occurrence.

Simple Explanation

We keep a "current champion" (the biggest number seen so far) and its position. As we scan, we only replace the champion when we find someone strictly bigger — a tie doesn't dethrone the current champion.

Dry Run

Array: [3, 8, 5, 8, 2]

i	arr[i]	arr[i] > arr[maxIndex]?	maxIndex
0	3	— (start)	0
1	8	8 > 3 → Yes	1
2	5	5 > 8 → No	1
3	8	8 > 8 → No (not strictly greater)	1

4	2	2 > 8 → No	1
---	---	------------	---

Output: 1

JavaScript Solution

```
function indexOfMax(arr) {
  let maxIndex = 0;
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > arr[maxIndex]) {
      maxIndex = i;
    }
  }
  return maxIndex;
}

console.log(indexOfMax([3, 8, 5, 8, 2])); // 1
```

Time Complexity

$O(n)$

Space Complexity

$O(1)$

Key Learning

Using strict `>` instead of `>=` is what preserves the "first occurrence" rule when duplicate maximum values exist.

Why `break` Matters (and When It Doesn't)

Why `break` is used for the First Occurrence

Once we scan left to right and find a match, it is mathematically guaranteed to be the earliest possible match. Continuing the loop cannot change the answer — it can only waste time. So we `break` to stop early and save computation.

Why `break` is NOT used for the Last Occurrence

The last occurrence can only be confirmed after checking every remaining element — a later match might still be waiting further in the array. If we `break` on the first match, we'd incorrectly return the first occurrence instead of the last.

Why `break` is NOT used for Counting Occurrences

Counting requires visiting **every** element to get an accurate total. Stopping early after the first (or any) match would produce an incomplete, incorrect count.

Rule of thumb: Use `break` only when finding the match early guarantees correctness. If you need information from the *entire* array (a total, or the *last* match), you must let the loop run to completion.

Why Functions Matter in DSA

Why Reusable Functions Are Useful

- They let the same logic be applied to different arrays and targets without rewriting code.
- They make testing easier — you can call the function with multiple inputs and verify outputs quickly.

- Interviewers expect clean, reusable functions rather than one-off scripts — it demonstrates good coding habits.
- They make code easier to read, debug, and extend later (e.g., adding edge case handling in one place).

Direct Code vs. Reusable Function Approach

Direct Code	Reusable Function
Values are hardcoded (e.g., searching only for 7)	Values are passed as parameters (target, arr)
Needs to be rewritten for every new input	Works for any input without changes
Harder to test	Easy to test with multiple calls
Looks like a one-time script	Looks like production-quality, interview-ready code

Example — Direct Code:

```
let arr = [1, 2, 3];
let found = false;
for (let i = 0; i < arr.length; i++) {
  if (arr[i] === 2) {
    found = true;
  }
}
```

Example — Reusable Function:

```
function contains(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return true;
  }
  return false;
}
```

The function version can be reused for **any** array and target — this is the pattern to build toward for every DSA problem going forward.

Day 2 Recap

Today's focus was Linear Search and the family of patterns built on top of it. Starting from a basic search, the same core loop was adapted to solve six related problems: finding an index, searching dynamically, counting matches, finding the first and last occurrence, and finding the index of the maximum value. The common thread across all of them is a single `for` loop with small variations in how `index`, `count`, or `maxIndex` are updated — and whether or not `break` is used.

Key Lessons Learned

- Linear Search checks each element sequentially and runs in $O(n)$ time.
- `-1` is the standard convention for "not found."
- `break` should only be used when stopping early cannot affect correctness.
- Small changes to the same base loop (`break` vs. `no break`, `>` vs. `>=`) produce entirely different, useful results.
- Passing values like `target` as function parameters makes code reusable and interview-ready.

Common Mistakes

- Using `break` in a "count occurrences" or "last occurrence" solution — this silently produces wrong answers.
- Using `>=` instead of `>` when tracking the maximum, which would return the *last* occurrence of the max instead of the first.
- Forgetting to initialize `index/count/maxIndex` before the loop,

- causing bugs or reference errors.
- Hardcoding the target value instead of accepting it as a parameter, making the function non-reusable.
- Not handling the "not found" case (forgetting to return `-1`).

Interview-Ready Explanation of Linear Search

"Linear Search is a sequential search algorithm that checks each element of an array one at a time, starting from the first index, until it either finds the target value or reaches the end of the array. It has a time complexity of $O(n)$ because in the worst case every element must be checked, and a space complexity of $O(1)$ since it uses no extra data structures. It's simple and works on both sorted and unsorted data, but it's less efficient than algorithms like Binary Search on large sorted datasets."

What I Should Practice / Revise

- Re-implement all six problems from scratch without looking at notes.
- Practice explaining out loud *why* `break` is or isn't used in each case.
- Try variations: find the index of the minimum value, find all indices of a target (return an array), search in an array of strings/objects.
- Get comfortable writing the base linear search loop from memory quickly.

Day 2 Completed ✓

Linear Search and its core variations (first occurrence, last occurrence, counting, max index) are understood and implemented in JavaScript with reusable functions. Ready to move forward to Day 3.